

S Lab Sync

Science Lab Management System — Project Report

Python Desktop Application (Tkinter) with MySQL Backend

---

1. What This Program Does

S Lab Sync is a desktop application built to help a school or college science lab keep track of everything happening in it: the students who use the lab, the teachers who supervise it, the equipment and chemicals on hand, and the experiments currently in progress. Instead of spreadsheets or paper registers, all of this information lives in a proper MySQL database and is managed through a single, consistent desktop interface. This is the Python edition of the app; it uses the same database layout as the Java Swing edition, so both can work with the same `lab_management` database.

Core screens:

Screen	Purpose
Overview	Live dashboard: counts of students, teachers, equipment, chemicals and experiments, plus a table of experiments showing the supervising teacher, status and how many students are taking part.
Students	Add, edit and delete student records (name, class/grade, roll number, phone).
Teachers	Add, edit and delete teacher records (name, subject, phone, email).
Equipment	Track lab instruments: name, category, quantity, location, condition and last maintenance date.
Chemicals	Track chemical inventory: name, formula, quantity, unit, storage location, expiry date and hazard level.
Experiments	Schedule an experiment, set its date, times, status and colour, choose the supervising teacher (by teacher ID), and assign any number of students to it.

2. What's Needed to Run It

The application is plain Python, so nothing has to be compiled. A few pieces do have to be in place before it will start and connect to a database.

Software / environment

- **Python 3.10 or newer, with Tkinter** — Tkinter is Python's built-in windowing toolkit. It is included with the python.org installers for Windows and macOS; on Linux it is a separate package (for example `python3-tk`).
- **MySQL Server** (5.7 or 8.x) running and reachable, normally on `localhost:3306`, with an account that is allowed to create databases and tables.
- **mysql-connector-python** — the MySQL driver for Python, installed with `pip install -r requirements.txt`.

Files in the project

- `main.py` — the application: windows, dialogs and pages.
- `db.py` — all MySQL access plus local settings storage.
- `theme.py` — the light and dark colour palettes and widget styling.
- `requirements.txt` / `requirements-dev.txt` — the packages needed to run the app, and to develop and test it.
- `run_slab_sync.bat` — a one-click launcher for Windows.
- `assets/`, `docs/`, `tests/` and `.github/` — the logo, these documents, the automated tests, and the continuous-integration workflow.

What happens automatically on first run

- The app asks for the MySQL host, username and password. If *Remember connection* is ticked, they are saved to `~/.labmanager/python_db.json`.
- It creates the `lab_management` database and all six tables itself — no manual SQL. Older databases that lack the `color_hex` column are upgraded automatically.
- The master login starts as `Master` / `7777` and can be changed from the sidebar; the new login is saved to `~/.labmanager/credentials.json`.

3. How It's Built (Architecture)

The code is split by responsibility rather than mixed into one file, which keeps each part easy to read and change. A schematic is in `docs/Architecture_Diagram.pdf`.

Part	Responsibility
main.py	The `App` class: start-up and MySQL setup dialog, master login, the sidebar, the six pages, and the add/edit, assign-students and change-password dialogs.
db.py	`LabDatabase` (connect, create schema, count, fetch, insert, update, delete, dashboard query, student assignment) plus `CredentialStore` and `ConfigStore` for the two local settings files.
theme.py	Two named colour palettes (light and dark) and the code that applies them to every Tkinter widget style, so each screen gets the right colours when it is built.
tests/	Automated tests: theme contrast, settings storage, database logic, the UI rendered in both themes, and database operations against a real MySQL server.
.github/workflows/	Runs the linter, the format check and all tests on every push and pull request, against a MySQL service, on Python 3.10 and 3.12.

4. What's Good About This Program

A polished, consistent interface

- A custom-styled interface rather than default Tkinter: a sidebar, colour-coded dashboard cards, styled tables, buttons and dialogs.
- A working **light / dark mode** toggle that applies to every screen and every pop-up dialog, including the login window.
- Experiments carry a colour that you can pick from a standard colour chooser.

Dark mode is protected by automated checks

- The test suite checks that every text colour meets a readable contrast ratio against its background in both themes (25 checks).
- It also renders every page and dialog in both themes and fails if any text would be invisible (15 checks), so unreadable-text bugs cannot slip back in unnoticed.

A smooth first-run experience

- A setup dialog guides the user through the MySQL connection instead of asking them to edit a config file, and can remember the connection for next time.
- The database and tables are created automatically, and older databases are upgraded in place.
- The master username and password can be changed from inside the app — no code editing.

Careful, safe data handling

- Values are always sent to MySQL as query parameters, never pasted into SQL, which protects against SQL injection. Table names are checked against a fixed whitelist.

- Changes are committed as a unit and rolled back if anything goes wrong; assigning students to an experiment is atomic.
- A damaged or incomplete settings file falls back to safe defaults instead of crashing the app at start-up.
- Deleting a teacher keeps their experiments (the teacher is simply cleared); deleting a student or experiment cleans up the matching assignments.

Tested and maintainable

- 67 automated tests, 11 of which run against a real MySQL server, covering creating, reading, updating and deleting every kind of record.
- Continuous integration runs the linter, formatter check and tests on every push, on two Python versions.
- Only three source files, with colours, database code and screens kept separate, so a contributor can find and change things quickly.
- Plain Python with one dependency, so it runs anywhere Python and Tkinter do (tested on Linux; designed with Windows in mind).

5. Things to Be Aware Of

- **Credentials are stored as plain text.** The MySQL password and the master password are kept in readable JSON files in the user's profile folder. The master login is a simple gate for a shared lab computer, not per-user authentication, so use a MySQL account with limited permissions and keep MySQL on a trusted network.
- **Teachers are chosen by ID number** in the experiment form, not from a drop-down list.
- **Platforms:** the app was developed for Windows and has been tested on Linux. On macOS the sidebar buttons may be drawn differently, because macOS ignores some Tkinter button colours; this has not been tested.

This report describes the current state of the S Lab Sync Python project, based on its source code and automated tests.